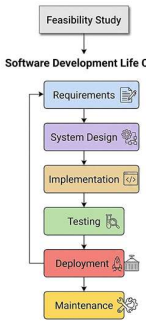
SE: Applying engineering principles to build software that is reliable, efficient & easy to maintain; Follows structured processes; Ensuring quality at every step; Planning across the entire software lifecycle; Saves time & money in the long run; Protects users & ensures their satisfaction; Avoids major failures; **Software Crisis (1960s – 1990s)**: Many software projects were failed due to over budget, missed deadlines, system crashes & disasters; **Therac-25**: Patients received massive radiation overdoses (poor testing, missing safety checks); **Ariane 5 Rocket**: Exploded 40s after launch (reused code w/o proper adaptation); **Mariner 1 (NASA)**: Veered off course & self-destructed (a missing hyphen in code!); **Knight Capital**: Lost $440 million in 30 mins (wrong code update in stock trading bot); **SDLC**: A step-by-step process used to develop software; Helps team meet user needs & expectations; Saves time & cost by planning ahead; Avoids risks & delivery delays; **Feasibility Study**: Used by managers/stakeholders to approve/reject the project; **Technical Feasibility**: Do we have the required hardware, software & skills? **Economic Feasibility**: Will the benefits outweigh the expenses? **Legal Feasibility**: Are there any laws, policies / regulations to follow? **Operational Feasibility**: Will the system work in the real world? Will users use it? **Schedule Feasibility**: Can we deliver it on time? **Requirement Gathering & Analysis**: Write it down clearly in a Software Requirements Specification (SRS) of what the user wants the system to do; **Involved people**: Clients; Users; Business analysts; **Techniques for gathering requirements**: **Interviews** (w. users); **Questionnaires**; **Observations** (watching their daily works to find pain points, inefficiencies/system improvement needs); **Document analysis** (current reports, forms/system manuals to understand existing workflows & rules); **Group discussions** (workshops/focus groups; resolves conflicts & builds shared agreement); **Showing examples** (prototyping – create mockups/sample screens to help users visualize the system & give early feedback); **Generating ideas** (brainstorming); **Functional Requirements**: Describe what the system should do; **Non-Functional Requirements**: Describe how well the system should perform (e.g.: *accessible on both desktop & mobile devices*); **Limitations (Constraints)**: Rules/boundaries the system must follow (e.g.: *deployed within the university's local network only*); **Best practices**: Involve all key stakeholders early in the process; Use clear, simple language (avoid technical jargon); Validate requirements w. stakeholders using reviews / walkthroughs (a peer review meeting where the team goes through the software requirements, design, or code step by step to check for issues); Clearly document any assumptions & limitations; Maintain traceability to track requirements through design, development & testing; **Common mistakes**: Guessing stakeholder needs (assuming what users want w/o asking/confirming); Using too much technical jargon (writing requirements that non-technical users can't understand); Not documenting assumptions/limitations; Ignoring non-functional requirements; Failing to update requirements & changes; **Stakeholder**: Any person or group who has an interest in the software system (uses/manages the system); **End Users**: Directly uses the system; **Internal Users**: Staff within the organization who support/manage the system; **External Partners**: 3rd parties that the system interacts with (payment gateways, suppliers); **Regulatory Bodies**: Authorities that set rules / standards the system must follow; **Project Sponsor/Client**: Funding/owning the system; **Requirement Validation**: Checks correctness w. stakeholders; **Traceability**: Maintains links through later lifecycle works; **Requirement Traceability**: Requirement through design, development & testing, helping the team verify that it's implemented & tested;

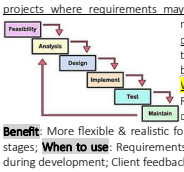


System Design: SRS (Software Requirements Specification) → System blueprint; Break down what the system should do → Design components; **What's Included**: UI Design; Database Schema; System Architecture (front-end, back-end, APIs together); **Output**: **High-level design** (overall structure); **Low-level design** (detailed logic & components); **Implementation (coding)**: Developers write actual code based on design documents; Follow coding standards, use the right tools & technologies; Built module by module; Each module is tested individually (unit testing); **Testing**: Tests both individual parts (modules) & the entire system; Goal is to find & fix bugs; Makes sure the software works as expected based on the requirements; **Types**: **Unit Testing** (function/module); **Integration Testing** (Makes sure separately developed modules work well together); **System Testing** (whole system); **Acceptance Testing** (Check if system meets client's requirements); **Deployment**: Finished software is installed in the real environment where users will use it; May include a pilot release (testing with a small group first); Users are trained & user manuals / help documents are prepared; Help desks, technical support/FAQs are provided to assist users during the transition; **Maintenance**: Bug fixes, feature updates & system improvements; The goal is to keep the system functional, useful, problem-free, secure & up to date; **Why It Matters**: New user needs; Technology updates; Security risks; **Types**: **Corrective Maintenance**: Fixes bugs/errors found after system is deployed; **Adaptive Maintenance**: Changes software to work in a new environment; **Perfective Maintenance**: Improves system by adding new features/enhancing performance; **Ethics**: Set of moral values that help us decide what is right/wrong in our actions; Being responsible, fair & professional in everything we do; It guides professionals throughout the entire SDLC; **Key Ethical Values**: **Responsibility** (take ownership of your work and its impact; public safety; proper escalation); **Transparency** (be honest about how the software works); **Privacy** (protect users' personal information); **Fairness** (avoid discrimination/bias in your system); **IEEE Code of Ethics**: Global body for engineering, computing & electronics; Publishes technical standards; **Focus**: Hardware, electrical & computer engineering; Last Updated 2020 (minor revision); **ACM (Association for Computing Machinery) Code of Ethics & Professional Conduct**: World's largest society for computing professionals; **Focus**: Computer science, education, software research; Offers journals, conferences & certifications; Last updated in 2018; **IEEE/ACM Code of Ethics**: A joint code created in 1999 specifically for Software Engineers; Known as the Software Engineering Code of Ethics & Professional Practice; Includes 8 core principles (public, client, product, judgment, management, profession, colleagues, self); **BCS Code of Conduct (UK)**: Focus on public interest, professional competence & accountability; **Australian Computer Society (ACS) Code of Ethics**: Includes privacy, respect, social impact; **IFIP (International Federation for Information Processing) Code of Ethics**: Promotes ethical computing globally;

Software Process Model: Structured method/strategy that guides how the SDLC phases are carried out; Provides framework/roadmap for organizing & managing tasks in SDLC;

SDLC	Software Process Models
Defines what phases to follow	Defines how to execute those phases
General structure	Specific implementation
Same phases used across models	Each model organizes, iterates, or emphasizes phases differently

'**Waterfall Model**: Linear; Step-by-step process; Each phase is completed before moving to the next; Simple & easy to understand; Each phase has clear goals & deliverables; Not suitable for projects where requirements may change later; One-time delivery **Best use case**: Project requirements are clearly known in advance & unlikely to change; '**Iterative Waterfall Model**: Modified version of traditional Waterfall Model; Allows feedback & revisions between phases; Follows same step-by-step phases as classic Waterfall, but teams can go back to earlier phases if needed; Reduces rigid structure of original Waterfall model; Allows for corrections & improvements before final product is delivered;

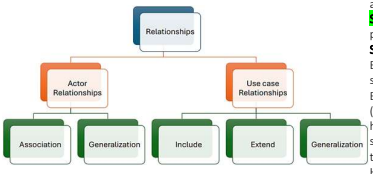


Benefit: More flexible & realistic for real-world projects where issues may only appear in later stages; **When to use**: Requirements are mostly clear, but some minor changes maybe needed during development; Client feedback is available @ the end of each phase; Medium-sized system w. moderate risk (not too complex, but not too simple either); '**V Model (Verification & Validation)**: Variation of waterfall; Focuses on testing @ every stage of development; **Left** → Planning & designing; For every development step, there is a corresponding test step; **Why use it**: This model helps catch errors early by planning tests as soon as each phase begins; **Common use**: Ideal for safety-critical systems (Medical devices, Automotive systems); '**Prototyping Model**: Initially build a quick & rough version of the system (a prototype) to show users early; They test it & give feedback, so we can improve the design before building the final system; **Throwaway Prototyping**: Temporary prototype built for learning & then discarded; **Evolutionary Prototyping**: Progressively refines the prototype into operational system; **How it works**: Gather basic requirements → Build a quick prototype → Show it to the user → Get feedback & improve it → Once users are happy → Build the real system; **Why use it**: Helps understand what user really wants; Saves time by avoiding big mistakes; Great for projects where requirements are unclear/may change; '**Incremental Model**: System is developed step-by-step in small parts called increments; Built & delivered in smaller functional units, not all at once; Each increment adds new features to the system; Every increment goes through the entire SDLC; User feedback from earlier increments can help improve future ones; **Why use it**: It allows early delivery of useful features & supports ongoing improvements; **When to use**: Working on very large projects; Most requirements are known, but full system needs to be developed in phases; Quick delivery of working features & early user feedback; To spread out the risk across multiple stages of development; Version based increments; '**Spiral Model**: Risk-focused & iterative; Waterfall + Prototyping; Each loop → Phase of the project; Project moves through these loops repeatedly, improving the product each time; **Structure**: **Planning**: Set goals, define system requirements; Identify constraints & possible solutions; **Risk Analysis**: Identify potential risks (cost, time); Analyze & take steps to reduce/eliminate those risks; **Engineering**: Design, build, & test part of the system (a prototype/increment); **Evaluation**: Customer reviews the work done; Based on feedback, next loop (iteration) is planned; **When to use**: Large, complex / high-risk project; Requirements aren't fully known @ the beginning & may change over time; Project needs frequent reviews, testing & updates; Want to identify & reduce risks early: Mistakes can be very costly/dangerous (defense/aerospace systems); '**Agile Model**: Iterative & incremental; Built & delivered in small usable parts called sprints (usually 1–4 weeks); Each sprint delivers a working product; Encourages continuous improvement & quick responses to change; Based on the Agile Manifesto (2001) – values people & interactions over rigid processes; **Focuses on**: Team collaboration; Customer involvement & feedback; Flexibility to change; **E.g.**: Mobile Applications; Cloud-based SaaS Products; **Best for**: Projects where requirements may change frequently & fast delivery is important; **Agile Frameworks**: It is a philosophy based on a set of values & principles (from Agile Manifesto); **Scrum**: Focuses on working in short, fixed-length cycles called sprints (usually 2–4 weeks); **Key Roles**: Product Owner; Scrum Master (guides the team); Team; **Main Elements**: Backlogs (lists of work to do); Increment (working part of the product); **Key Meetings**: Sprint Planning, Daily Stand-up, Sprint Review, Retrospective; **Kanban**: Focuses on visualizing tasks on a board & managing continuous flow of work; Each framework helps teams stay flexible, collaborate & deliver faster; **Factors to choose the model**: **Fixed vs. Changing Requirements**: Stable requirements → Waterfall; Changing needs → Agile/Spiral; **Project Size & Complexity**: Large, complex systems → Spiral/V-Model for structure & risk control; **Risk Level**: High-risk projects need risk assessment → Spiral; **Time to Market Requirements**: Need fast delivery? → Agile, Scrum or Incremental; **Team Style**: Structured teams → Scrum; Flexible teams → Kanban; **User Feedback**: If frequent feedback is critical →Agile-based models;

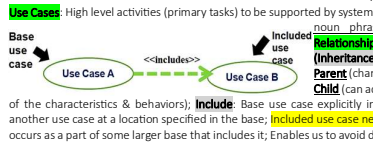
Model	Approach	Delivery Style	Best For	Drawbacks
Waterfall	Linear	One final delivery	Simple, well-defined projects	No feedback until the end
Iterative Waterfall	Phased + Feedback	One final delivery	Moderate complexity projects	Still limited flexibility
V-Model	Validation-based	One final delivery	Safety-critical systems (e.g. avionics)	Costly to fix late changes
Prototyping	Build quick model, get feedback, refine	Evolves through user feedback	Unclear or changing requirements	Prototype may be mistaken as final; scope creep
Incremental	Phased	Partial releases	Evolving requirements	Requires stable architecture upfront
Spiral	Risk-driven	Incremental	High-risk, large-scale projects	Complex and expensive
Scrum (Agile)	Sprint-based	Every sprint	Teams needing structure and roles	Rigid schedules and roles
Kanban (Agile)	Flow-based	Continuous	Maintenance and support workflows	May lack clear deadlines

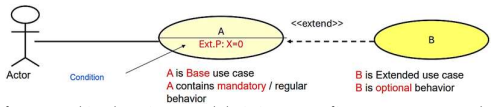
Agile Development: Set of methods & practices based on Values & Principles expressed in Agile Manifesto → 4 key Values (defines core philosophy of Agile) & 12 Principles (guides on how these values are applied in practice); Handles changing requirements more effectively; **Traditional Development**: Cost ↑; Changes are not acceptable; Can detect errors only in the latter part of the SDLC; Less/no iterations; Lack of transparency; Time consuming; Rework required; Customer may not get what they need; Requirements may change; **How Agile Helps**: Delivers software in small increments; Encourages regular customer feedback; Adapts to changing requirements; Identifies issues early; Focuses on delivering high-value features first; Helps teams deliver better software faster while responding to change; **4 Key Values**: Individuals & Interactions; Working Software; Customer Collaboration; Responding to Change; **Agile Methodologies**: Provides overall structure for managing a project; **Scrum**: Work is organized into Sprints; **Kanban**: Work is managed using a visual board; **Extreme Programming (XP)**: Focuses on engineering practices & code quality; **Lean Software Development**: Focuses on eliminating waste & maximizing value; **Agile Practices**: Specific techniques used by development team during implementation; **Test-Driven Development (TDD)**: Write tests before writing code; **Pair Programming**: 2 developers work together on same task; **Behavior-Driven Development (BDD)**: Defines system behavior using business-focused scenarios; **Scrum**: Divides work into short iterations → Sprints; Working software at the end of each Sprint; Feedback collected regularly; Helps teams work collaboratively & adapt to change; Consists of Roles, Artifacts & Activities; '**Roles**: **Product Owner**: Represents customers & stakeholders; Define & prioritize requirements; Maintain Product Backlog; Communicate customer needs to team; Decide what features should be developed; Accept/reject completed work; Maximizing value of product; **Scrum Master**: Ensures Scrum practices are followed & supports team; *facilitator, not a traditional project manager*; Remove obstacles that affect progress; Promote collaboration & communication; Helping the team work effectively; **Developers**: Build & deliver product; Design, develop & test software; Complete Sprint Backlog items; Collaborate w. other team members; Maintain product quality; Deliver a working product increment at the end of each Sprint; '**Artifacts**: To provide transparency & help teams manage their work; Helps Scrum Team understand what needs to be done & how much progress has been made; '**Product Backlog**: Prioritized list of requirements, features & improvements for product; Work is prioritized; Items are placed in correct sequence; Highest-value items are completed first; Changes & evolves throughout project; Product Owner is responsible for creating & maintaining; ID (PBI-1…), User Story, Priority (High, Medium, Low); **Ways to store**: Collection of index cards/post-it on the wall; Excel; User Stories; **User Stories**: Concise description of a Functionality that will be valuable to users of system; Convenient format for expressing desired business value for many types of product backlog items; Understandable to both business & technical people; As a *<user role>* I want to *<goal>* so that *<benefit>*; **Good User Story**: Clearly identify user role; Avoid technical implementation details; **Selection Criteria**: Highest Business Value; Core System Functionality; Foundation for Future Features (other features should depend on this core function); Deliver a Working Product Early (users could be able to use basic system); **Testable Acceptance Criterion**: Makes its boundaries & expected behavior testable; Provides observable Given-When-Then acceptance criterion w/o prescribing implementation; '**Sprint Backlog (release backlog)**: List of user stories identified by Scrum team to be completed during sprint; Subset of product backlog; Work is performed in iterations/cycles of up to a calendar month → sprints; Before starting Sprint, team should come up w. Sprint backlog; A **to-do item** can belong to sprint backlog; Task ID (T-1.1…), Task, Est. Hours; '**Burn Down Charts**: Graphical representation of work left to do (backlog) vs. time; Useful for predicting when all of the work will be completed; '**Activities**: During each sprint, plan, monitor, review & improve their work; '**Sprint Planning**: Decide what work will be completed during Sprint; Conducted at beginning of each Sprint; Sprint ID (SP1-01…), User Story, Priority; Always develop highest-value features 1st & deliver working software in small, incremental releases; **Purpose**: Select Product Backlog items for Sprint; Define Sprint Goal; Create Sprint Backlog; '**Daily Scrum**: *Monitor progress & discuss obstacles*; Short daily meeting (typically 15mins) held by Scrum Team to monitor progress; **Key Questions**: What did I complete since the last Daily Scrum? What will I work on next? Are there any obstacles blocking my progress? '**Sprint Review**: Conducted at end of Sprint; *Demonstrate completed work*; Gather stakeholder feedback; Review progress toward Sprint Goal; Update Product Backlog if needed; Inspect product outcomes w. stakeholders; '**Sprint Retrospective**: Reflect on team's performance; Reflect on Sprint; Identify what went well; Identify areas for improvement; *Plan improvements for next Sprint*;

Requirement Specification: After document requirements are identified, **we use graphical notations**: **Use Case Diagrams**: Represents proposed functionality of new system; Captures functional requirements; Helps to demonstrate high-level behavior; Supports requirement specification; Act as means of communicating **w. stakeholders** about what the system is intended to do; Can be used to communicate to management, customers & other non-development people; It focuses on WHAT a system will do when it is completed (but not HOW a system will do anything); **Components**: **System**: Something that performs function(s); **System Boundary**: Boundary between system & actors; **Actors**: External entities; Roles (human users, external hardware, other systems) adopted by those participating; Have direct interactions;



Use Cases: High level activities (primary tasks) to be supported by system; Unit of behavior; Verb-noun phrase (reserve a room); **Relationships / Links**: **Generalization (Inheritance)**: Unfilled Arrow; **Parent** (characteristic & behavior) ← **Child** (can add, modify, or ignore any of the characteristics & behaviors); **Include**: Base use case explicitly incorporates behavior of another use case at a location specified in the base; Included use case never stands alone; It only occurs as a part of some larger base that includes it; Enables us to avoid describing the same flow of events several times by putting common behavior in a use case of its own; Arrow targets reused mandatory use case; **Extend**: Base use case implicitly incorporates behavior of another use case at certain points → Extension points; Base use case may stand alone, but under certain conditions, its behavior may be extended by behavior of another use case; Arrow targets complete base use case;





Number	*Unique use case number*	
Name	*Brief noun-verb phrase*	
Summary	*Brief summary of use case major actions*	
Priority	*1-5 (1 = lowest priority, 5 = highest priority)*	
Preconditions	*What needs to be true before use case "executes"*	
Postconditions	*What will be true after the use case successfully "executes"*	
Primary Actor(s)	*Primary actor name(s)*	
Secondary Actor(s)	*Secondary actor name(s)*	
Trigger	*The action that causes this use case to begin*	
Main Scenario	**Step**	**Action**
	Step #	*This is the "main success scenario" or "happy path."*
	…	*Description of steps in successful use case "execution"*
	…	*This should be in a "system-user-system, etc." format.*
Extensions	**Step**	**Branching Action**
	Step #	*Alternative paths that the use case may take*
Open Issues	*Issue #*	*Issues regarding the use case that need resolution*

Use Case Scenarios: **Scenario**: Formal description of the flow of events that occur during execution of a Use Case instance; Defines specific sequence of events between system & external Actors; **Main scenario**: Describes what happens when everything goes to plan; It's written under assumption that everything is okay, no problems occur & it leads directly to desired outcome of the use-case; **Other scenarios**: Describes what happens when variations to Main scenario arise, often leading to different outcomes; **Flow of events should include**: How & when use case starts & ends; When use case interacts w. actors; What objects are exchanged; Basic flow & Alternative flows (exceptional) of behavior;

Activity Diagrams: Dependencies between activities; Represents Workflows; **When to use?** Modeling business processes; Analyzing system functionality to identify the use cases; Analyzing individual use cases in detail; Clarifying concurrency issues; **Initial/Start Node**: Filled Circle; **Final Node**: Within circle; **Action Node**: Rounded rectangle; Task which needs to be done (verb phrase in present tense); Sequence of actions; **Transition (control flow/directed flow/edge)**: Arrow; **Decision Node**: Diamond; [Guard expression] for each possibility; No limit on the # of branches; **Merge Node**: Diamond; To merge the flow of control from branches back together; **Call action/Sub Activity**: Rake symbol within action node; An activity that is defined in more detail on another activity diagram; **Forks & Joins**: Thick Bar (starting & ending); Activities that can occur in parallel; Order of execution of the activities can be whatever is convenient for the implementation; Every fork must have a join; **Thread**: Flow of control; **Iteration**: Asterisk; Alt. we can use a decision node to represent an iteration; Guard conditions should be mentioned; **Partitioning**: An activity group for actions that have some common characteristic; It corresponds to organizational units/business actors in a business model; **Swim lanes**: **Sub-partitioning** (e.g. Order department is a subclass of Department class);